



INSTRUCTOR: ILSE ALINE
MORALES CASTILLO

Pruebas

Introducción a las Pruebas Unitarias

¿Qué son las pruebas unitarias?

Son pruebas automáticas que verifican el comportamiento de una unidad pequeña del código (generalmente un método).
Ayudan a identificar errores temprano y a asegurar que el código sigue funcionando como se espera.

¿Por qué son importantes las pruebas unitarias?

Detectan errores rápidamente. Mejoran la calidad y mantenibilidad del código.
Facilitan el refactorizado sin introducir nuevos errores.

Herramientas para Pruebas Unitarias en Java



JUnitFramework de pruebas unitarias más popular para Java.Ofrece anotaciones como @Test, @Before, @After, @BeforeClass, etc.



MockitoFramework de mocking que permite crear objetos simulados (mocks) para pruebas unitarias.

Configuración de JUnit y Mockito

- Uso de dependencias

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.7.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>3.9.0</version>
  <scope>test</scope>
</dependency>
```

Estructura Básica de una Prueba con JUnit

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    @Test
    void testSuma() {
        Calculadora calculadora = new Calculadora();
        int resultado = calculadora.sumar(2, 3);
        assertEquals(5, resultado);
    }
}
```

```
import static org.mockito.Mockito.*;

class ServicioTest {

    @Test
    void testServicio() {
        // Crear mock
        Servicio servicioMock = mock(Servicio.class);

        // Configurar comportamiento
        when(servicioMock.obtenerDatos()).thenReturn("Datos simulados");

        // Usar el mock
        String resultado = servicioMock.obtenerDatos();

        // Verificar el resultado
        assertEquals("Datos simulados", resultado);
    }
}
```

Mockito: ¿Qué es y cómo funciona?

Mockito

- Permite simular objetos y comportamientos para evitar dependencias en pruebas unitarias.
- Útil cuando dependemos de clases externas, bases de datos, o servicios web.

Usando Mockito con Dependencias

- inyectar dependencias en una clase y mockearlasSe puede usar @Mock para crear mocks e @InjectMocks para inyectar estos en la clase que se va a probar.

```
import org.mockito.*;
import static org.mockito.Mockito.*;

class ServicioTest {

    @Mock
    private Repositorio repositorioMock;

    @InjectMocks
    private Servicio servicio;

    @Test
    void testServicioConMock() {
        when(repositorioMock.obtenerDatos()).thenReturn("Datos desde mock");

        String resultado = servicio.obtenerDatos();

        assertEquals("Datos desde mock", resultado);
    }
}
```

Resumen y Buenas Prácticas

01

Escribe pruebas pequeñas y claras.

02

No dependas de la red ni bases de datos reales en pruebas unitarias.

03

Usa mocks para las dependencias externas.

04

Asegúrate de que las pruebas sean fáciles de ejecutar y manténlas actualizadas.